

DBMS – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Project Title : Lenskart – E-Commerce Order Management System

Objectives

1. Create Review and Rating tables
2. Store customer feedback and ratings.
3. Retrieve product review details.
4. Calculate average product ratings using aggregate functions.
5. Identify highly rated products.

1. Database Design

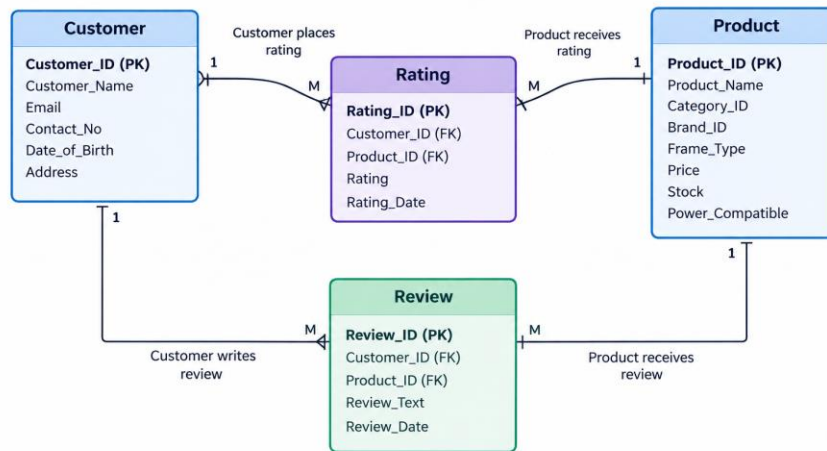
1.1 Review Table

Attribute	Key Type
Review_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Review_text	-
Review_date	-

1.2 Rating Table

Attribute	Key Type
Rating_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	Not Null
Rating_Date	-

2. Table Relationships



3. Table Creation

3.1 Create Review Table

```
CREATE TABLE Review (  
    Review_ID NUMBER PRIMARY KEY,  
    Customer_ID NUMBER,  
    Product_ID NUMBER,  
    Rating NUMBER CHECK (Rating BETWEEN 1 AND 5),  
    Review_Text VARCHAR2(500),  
    Review_Date DATE,  
    CONSTRAINT fk_review_customer  
        FOREIGN KEY (Customer_ID)  
        REFERENCES Customer(Customer_ID),  
    CONSTRAINT fk_review_product  
        FOREIGN KEY (Product_ID)  
        REFERENCES Product(Product_ID)  
);  
Table created.
```

3.2 Create Rating Table

```

CREATE TABLE Rating (
    Rating_ID NUMBER PRIMARY KEY,
    Customer_ID NUMBER,
    Product_ID NUMBER,
    Rating NUMBER NOT NULL,
    Rating_Date DATE,
    CONSTRAINT fk_rating_customer
        FOREIGN KEY (Customer_ID)
        REFERENCES Customer(Customer_ID),
    CONSTRAINT fk_rating_product
        FOREIGN KEY (Product_ID)
        REFERENCES Product(Product_ID)
);
Table created.

```

4. Review Data Insertion

```

INSERT INTO Review
VALUES (1, 1, 201, 5, 'Excellent quality and comfortable to v
    DATE '2026-09-15');

1 row created.

INSERT INTO Review
VALUES (2, 2, 202, 4, 'Good product and stylish design',
    DATE '2026-09-16');

1 row created.

INSERT INTO Review
VALUES (3, 3, 203, 5, 'Very good quality glasses',
    DATE '2026-09-17');

INSERT INTO Review
VALUES (4, 4, 204, 3, 'The product is good but delivery was lat
    DATE '2026-09-18');

1 row created.

INSERT INTO Review
VALUES (5, 5, 205, 4, 'Useful for computer work',
    DATE '2026-09-19');

```

```
INSERT INTO Review
VALUES (6, 6, 201, 5, 'Excellent frame and good quality',
       DATE '2026-09-20');

1 row created.

INSERT INTO Review
VALUES (7, 7, 202, 4, 'Comfortable and attractive design',
       DATE '2026-09-21');

1 row created.

INSERT INTO Review
VALUES (8, 8, 203, 5, 'Very satisfied with this product',
       DATE '2026-09-22');

1 row created.

INSERT INTO Review
VALUES (9, 9, 204, 3, 'Average product but worth the price',
       DATE '2026-09-23');

1 row created.

INSERT INTO Review
VALUES (10, 10, 205, 4, 'Good glasses for daily use',
       DATE '2026-09-24');

1 row created.
```

5. Rating Data Insertion

```
INSERT INTO Rating
VALUES (1, 1, 201, 5, DATE '2026-09-15');

1 row created.

INSERT INTO Rating
VALUES (2, 2, 202, 4, DATE '2026-09-16');

1 row created.

INSERT INTO Rating
VALUES (3, 3, 203, 5, DATE '2026-09-17');

1 row created.

INSERT INTO Rating
VALUES (4, 4, 204, 3, DATE '2026-09-18');

1 row created.

INSERT INTO Rating
VALUES (5, 5, 205, 4, DATE '2026-09-19');

1 row created.
```

```

INSERT INTO Rating
VALUES (6, 6, 201, 5, DATE '2026-09-20');

1 row created.

INSERT INTO Rating
VALUES (7, 7, 202, 4, DATE '2026-09-21');

1 row created.

INSERT INTO Rating
VALUES (8, 8, 203, 5, DATE '2026-09-22');

1 row created.

INSERT INTO Rating
VALUES (9, 9, 204, 3, DATE '2026-09-23');

1 row created.

INSERT INTO Rating
VALUES (10, 10, 205, 4, DATE '2026-09-24');

1 row created.

```

6. Retrieve Product Review Details

```

SELECT
    r.Review_ID,
    r.Customer_ID,
    r.Product_ID,
    p.Product_Name,
    r.Rating,
    r.Review_Text,
    r.Review_Date
FROM Review r
JOIN Product p
    ON r.Product_ID = p.Product_ID;

```

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID	PRODUCT_NAME	RATING	REVIEW_TEXT	REVIEW_DA
1	1	201	Classic Eyeglasses	5		

```

REVIEW_ID CUSTOMER_ID PRODUCT_ID

```

PRODUCT_NAME				
RATING				
REVIEW_TEXT				
REVIEW_DA				
Excellent quality and comfortable to wear				
15-SEP-26				
REVIEW_ID CUSTOMER_ID PRODUCT_ID				
PRODUCT_NAME				
RATING				
REVIEW_TEXT				
REVIEW_DA				
		2	2	202
Stylish Sunglasses				
		4		
REVIEW_ID CUSTOMER_ID PRODUCT_ID				

REVIEW_ID CUSTOMER_ID PRODUCT_ID				
PRODUCT_NAME				
RATING				
REVIEW_TEXT				
REVIEW_DA				
		5	5	205
Blue Light Glasses				
		4		
REVIEW_ID CUSTOMER_ID PRODUCT_ID				
PRODUCT_NAME				
RATING				
REVIEW_TEXT				
REVIEW_DA				
Useful for computer work				
19-SEP-26				

REVIEW_ID	CUSTOMER_ID	PRODUCT_ID

PRODUCT_NAME		

RATING		

REVIEW_TEXT		

REVIEW_DA		

Very satisfied with this product		
22-SEP-26		

REVIEW_ID CUSTOMER_ID PRODUCT_ID		

PRODUCT_NAME		

RATING		

REVIEW_TEXT		

REVIEW_DA		

	9	9 204
Contact Lenses		
	3	

PRODUCT_NAME		

RATING		

REVIEW_TEXT		

REVIEW_DA		

	10	10 205
Blue Light Glasses		
	4	

REVIEW_ID CUSTOMER_ID PRODUCT_ID		

PRODUCT_NAME		

RATING		

REVIEW_TEXT		

REVIEW_DA		

Good glasses for daily use		
24-SEP-26		

10 rows selected.		

7. Calculate Average Product Ratings

This uses the SUM() aggregate function.

```
SELECT SUM(Rating) AS Total_Ratings
FROM Review;
```

TOTAL_RATINGS

TOTAL_RATINGS		
		42

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SELECT
    p.Product_ID,
    p.Product_Name,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Product p
JOIN Review r
    ON p.Product_ID = r.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
HAVING AVG(r.Rating) >= 4;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

	201		
	Classic Eyeglasses		
			5
	202		
	Stylish Sunglasses		
			4

PRODUCT_ID			

PRODUCT_NAME			

AVERAGE_RATING			

	203		
Reading Glasses			
		5	
	205		
Blue Light Glasses			
PRODUCT_ID			

PRODUCT_NAME			

AVERAGE_RATING			

		4	

9. Product Rating Report

A combined report can be generated using the Rating and Product tables.

```

SELECT
    p.Product_ID,
    p.Product_Name,
    COUNT(r.Rating_ID) AS Total_Ratings,
    ROUND(AVG(r.Rating), 2) AS Average_Rating,
    MIN(r.Rating) AS Minimum_Rating,
    MAX(r.Rating) AS Maximum_Rating
FROM Product p
JOIN Rating r
    ON p.Product_ID = r.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
ORDER BY
    Average_Rating DESC;

```

PRODUCT_ID				

PRODUCT_NAME				

TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING	

	201			
Classic Eyeglasses				
	2	5	5	5
	203			

203	Reading Glasses	2	5	5	5
PRODUCT_ID					
PRODUCT_NAME					
TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING		
205	Blue Light Glasses	2	4	4	4
202	Stylish Sunglasses				
PRODUCT_ID					
PRODUCT_NAME					
TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING		
2		4	4	4	
204					

203	Reading Glasses	2	5	5	5
PRODUCT_ID					
PRODUCT_NAME					
TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING		
205	Blue Light Glasses	2	4	4	4
202	Stylish Sunglasses				
PRODUCT_ID					
PRODUCT_NAME					
TOTAL_RATINGS	AVERAGE_RATING	MINIMUM_RATING	MAXIMUM_RATING		
2		4	4	4	
204					

204	Contact Lenses	2	3	3	3